TURING MACHINE

A Turing machine(TM) is a computer model that reads and writes to an infinite tape to perform computations. It was proposed by Alan Turing in 1936. The tape is divided into cells into which input is given. It contains a reading head that reads the input tape.

Formal definition of Turing machine

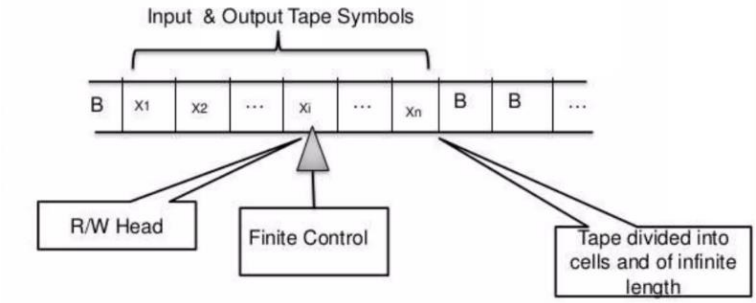
A TM is defined as a **7-tuple (Q, Σ, Γ, δ, q0, B, F)** where −

1. Q represents a finite set of states
2. Γ Finite set of allowable tape symbols
3. ∑ is the input alphabet
4. δ is the transition function; δ : Q × Γ → Q × Γ × {Left_shift, Right_shift}.
5. q0 is the initial state
6. B represents the blank symbol
7. F is the set of final states

δ is the transition function; δ : Q X Γ → Q X Γ X {L, R}.

This means that from one state, by getting one input from the input tape, the machine moves to a state. It writes a symbol on the tape and moves to left, right, or halts.

The following representation can be used to model the turning machine.



- The input tape has an infinite number of cells, each containing one input symbol, allowing the input string to be taped. The empty tape is filled with blank characters as shown in Fig.
- The tape head and the finite control are responsible for reading the current input symbol. The tape head moves from left to right.
- It consists of a finite set of states that a machine must pass through.
- External symbols are a finite set of symbols used to build the logic of a Turing machine.

Following are all possible results after performing all operations on input tape.

- HALT and ACCEPT: We reach an accepted state when we have done all operations.
- HALT and REJECT: We get a rejected state after doing all operations.
- LOOP: it means the machine fails to HALT or reject

Note: HALT is also called STOP

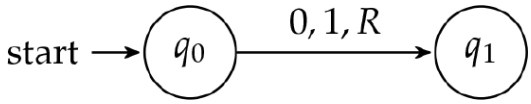
Turing machine are similar to Finite automata but some important differences are given below,

- A Turing machine can both write on the tape and read from it
- The read-write head can move both to the left and to the right.
- The tape memory is infinite.
- The special states for rejecting and accepting take immediate effect.

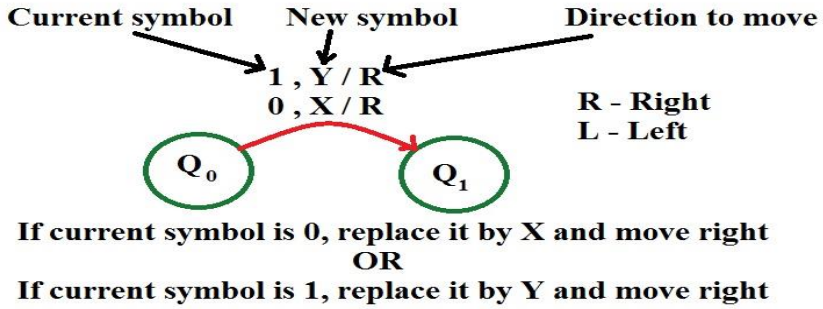
➢ **Representing Turing Machine**

Turing machines can be represented visually by *state diagrams*. The diagrams are composed of nodes connected by arrows. Each node represents a state of the machine. Each arrow represents an instruction that can be carried out from that state, with the specifics of the instruction written above or below the appropriate arrow.

Consider the following machine, which has only two internal states, q0q0 and q1q1, and one instruction:



The Turing machine has a read/write head and a tape with the input written on it. The instruction can be read as *if reading a* 0 *in state* q0*, write a* 1*, move right, and move to state* q1. This is equivalent to the transition function mapping ⟨q0,0⟩ to ⟨q1,1,R⟩.



➢ **Language of Turing Machine**

- The languages that are recognized by Turing Machines are also called Turing-recognizable languages, recursive enumerable languages, or semi-decidable languages.
- A language is recursive (or decidable) if there is a Turing Machine that halts and decides whether a given string is in the language or not.
- The language of a Turing machine M, denoted $\mathcal{L}(M)$, is the set of all strings that M accepts: $\mathcal{L}(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \}$ For any $w \in \mathcal{L}(M)$, M accepts w. For any $w \notin \mathcal{L}(M)$, M does not accept w. M might reject w, or it might loop on w.

A Turing Machine halts if it either accepts or rejects. For any given input, a TM may loop, accept or reject. So there is certainly a difference between halting and accepting, viz. accepting always implies halting but not vice versa.

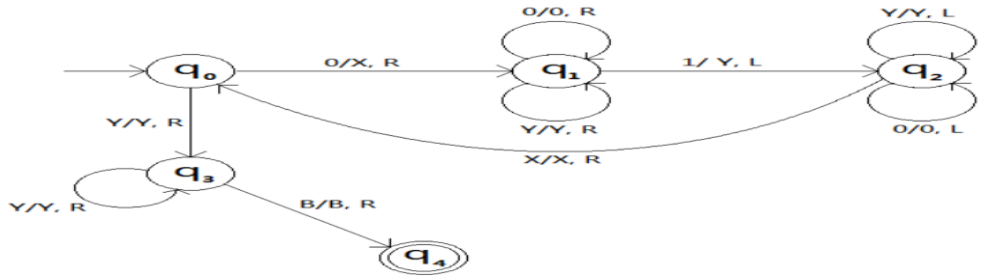
➢ **Instantaneous Descriptions for Turing Machines**

An Instantaneous Description (ID) of a Turing Machine is a snapshot of its current configuration, representing the state, head position, and tape contents at any given moment. It is conventionally written as $X_1X_2\ldots X_{i-1}\, q\, X_i\, X_{i+1}\ldots X_n$ where q is the current state, Xi is the symbol under the head, and $X_1\ldots.Xn$ represents the non-blank tape content.

The Instantaneous Description (ID) of a Turing machine remembers the following at a given instance of time −

- The contents of all the cells of the tape, starting from the rightmost cell up to at least the last cell, containing a non-blank symbol and containing all cells up to the cell being scanned.
- The cell currently being scanned by the read-write head.
- The state of the machine.

Example 1 : TM for L = { 0^n1^n | n>0} and IDs for string '0011'



Formal Definition:

The Turing Machine M is defined as a 7-tuple: M = (O, Σ, Γ, δ, q0, B, F)

Q (Finite set of states): {q0, q1, q2, q3, q4}

Σ (Input alphabet): {0, 1}

Γ (Tape alphabet): {0, 1, X, Y, B} (X for marked O, Y for marked 1, B for blank)

8 (Transition function): See Transition Table below

q0 (Start state): q0

B (Blank symbol): B

F (Final/Accepting states): {q4}

Transition Table

	0	1	X	Y	B
q_0	(q_1, X, R)	-	-	(q_3, Y, R)	-
q_1	(q_1, 0, R)	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	(q_2, 0, L)	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, R)
q_4	-	-	-	-	-

Instantaneous Descriptions (IDs for 0011)

Here is an example of an accepting computation by M. Its input is 0011. Initially, M is in state q_0, scanning the first 0, i.e., M's initial ID is $q_0 0011$. The entire sequence of moves of M is:

$$q_0 0011 \vdash Xq_1 011 \vdash X0q_1 11 \vdash Xq_2 0Y1 \vdash q_2 X0Y1 \vdash$$
$$Xq_0 0Y1 \vdash XXq_1 Y1 \vdash XXYq_1 1 \vdash XXq_2 YY \vdash Xq_2 XYY \vdash$$
$$XXq_0 YY \vdash XXYq_3 Y \vdash XXYYq_3 B \vdash XXYYBq_4 B$$

For another example, consider what M does on the input 0010, which is not in the language accepted.

$$q_0 0010 \vdash Xq_1 010 \vdash X0q_1 10 \vdash Xq_2 0Y0 \vdash q_2 X0Y0 \vdash$$
$$Xq_0 0Y0 \vdash XXq_1 Y0 \vdash XXYq_1 0 \vdash XXY0q_1 B$$

The behavior of M on 0010 resembles the behavior on 0011, until in ID $XXYq_1 0$ M scans the final 0 for the first time. M must move right, staying in state q_1, which takes it to the ID $XXY0q_1 B$. However, in state q_1 M has no move on tape symbol B; thus M dies and does not accept its input. □

Example 2

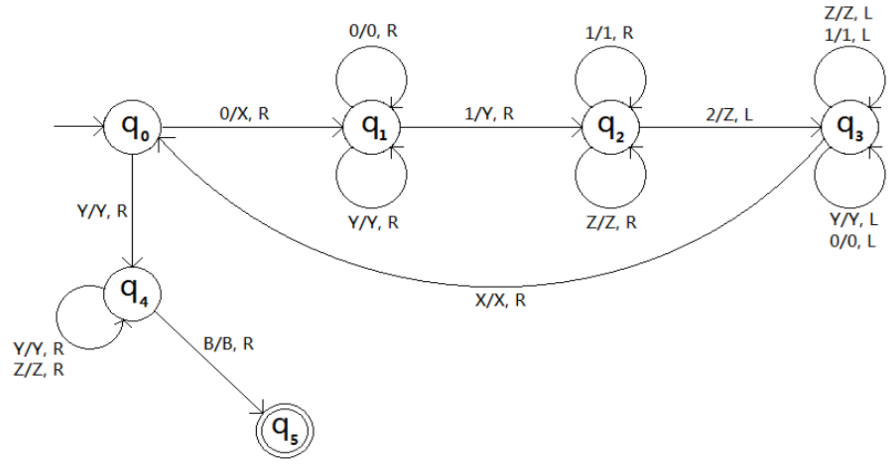


Figure 4: Turing machine accepts $\{0^n1^n2^n : n \geq 1\}$

Example3

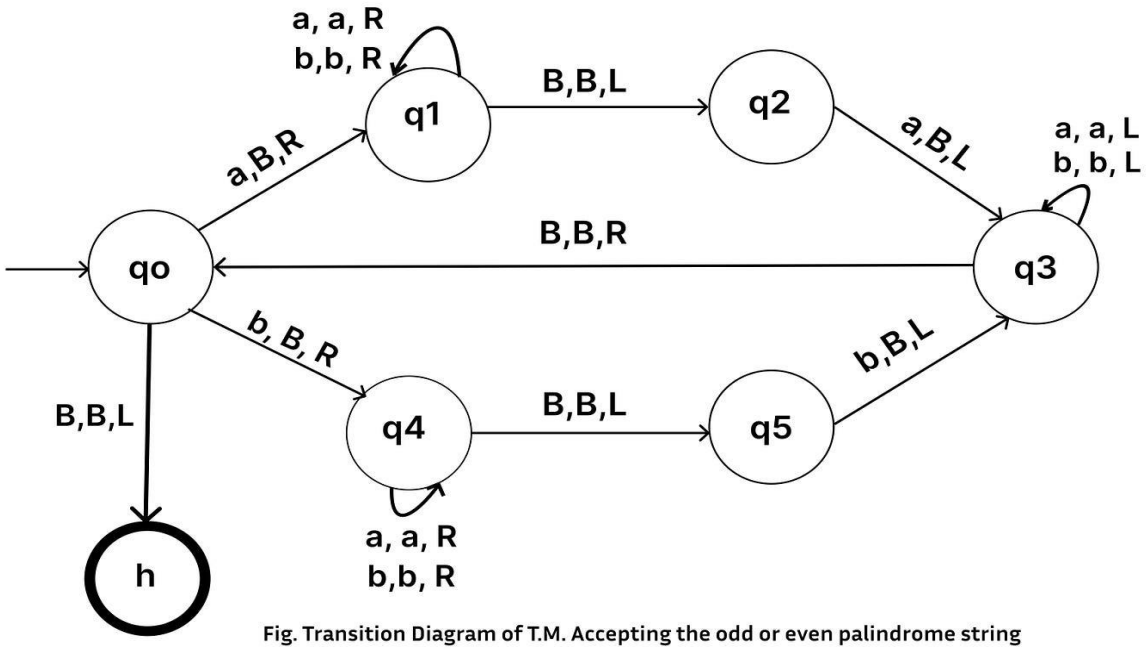


Fig. Transition Diagram of T.M. Accepting the odd or even palindrome string

Decidable Problems A problem is decidable if we can construct a Turing machine which will halt in finite amount of time for every input and give answer as 'yes' or 'no'. A decidable problem has an algorithm to determine the answer for a given input.

Examples

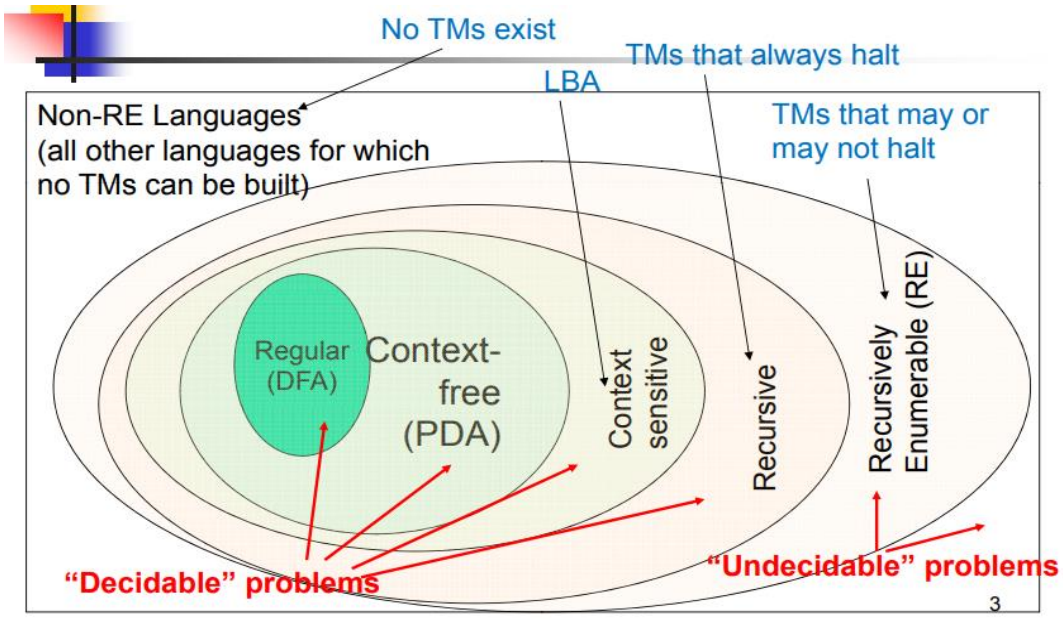
Equivalence of two regular languages, Finiteness of regular language, Emptiness of context free language

Undecidable Problems A problem is undecidable if there is no Turing machine which will always halt in finite amount of time to give answer as 'yes' or 'no'. An undecidable problem has no algorithm to determine the answer for a given input.

Examples

Ambiguity of context-free languages, Equivalence of two context-free languages

Semi-decidable Problems A semi-decidable problem is subset of undecidable problems for which Turing machine will always halt in finite amount of time for answer as 'yes' and may or may not halt for answer as 'no'. Relationship between semi-decidable and decidable problem



Recursive Languages (REC)

If L is recursive then L=L(M) for some TM M, and

If 'w' is in L then M halts in a final (accepting) state.

If 'w' is not in L. then M halts a non-final (non-accepting) state.

A TM of this type corresponds to our informal notion of an algorithm a well defined sequence of steps that always finishes and produces an answer. If we think of the language L as a problem as will be the case frequently then problem L is called decidable if it is a recursive language and it is called undecidable if it is not a recursive language

REC languages are also called as Turing decidable languages.

– If L is a recursive language, then a TM M that accepts L and always halts can be pictorially represented by a "chip" that has one input and two outputs.



Recursively Enumerable Languages (RE) are also called Recognizable because a Turing Machine can recognize a string in the language (accept it). It might not be able to decide if a string is not in the language since the machine might loop for that input.

If L is r.e. then L = L(M) for some TM M, and

If w is in L then M halts in a final (accepting) state.

If w is not in L. then M may halt in a non-final (non-accepting) state, or loop forever.

It means TM can loop forever for the strings which are not a part of the language. RE languages are also called as Turing recognizable languages.

– If L is a recursively enumerable language, then a TM M that accepts L can be pictorially represented by a "chip" that has one output.



– Conceivably, M could be provided with an output for "no," but this output cannot be counted on. Consequently, we simply ignore it.

Recursively Enumerable Languages produces any of 3 states : Accept & halt , Reject & halt , loop

Recursive Languages (REC) produces 2 states : Accept & halt , Reject & halt

Notes:

The set of all recursive languages is a subset of the set of all recursively enumerable languages.

Terminology is easy to confuse: A TM is not recursive or recursively enumerable, rather a language is recursive or recursively enumerable.

Properties of Recursive language

Decidability:

A recursive language is decidable, meaning there exists a Turing machine that will halt and decide membership for any string in finite time.

Halting Guarantee:

Unlike recursively enumerable languages, a Turing machine for a recursive language never loops forever; it always halts in an accept or reject state.

Subset of RE:

Every recursive language is also a recursively enumerable (RE) language, but not all RE languages are recursive.

Closure Properties:

- Complement: If language L is recursive, its complement L is alsc recursive.
- Union: If L_1 and L2 are recursive, L_1 U L_2 is recursive.
- Intersection: If L_1 and L2 are recursive, $L_1 \cap L_2$ is recursive.
- Concatenation & Kleene Star: Recursive languages are closed under concatenation (L1L2) and Kleene star (L*).
- Relationship to RE: A language is recursive if and only if both the language and its complement are recursively enumerable.

Examples:

Context-sensitive languages and regular languages are examples of recursive languages.

Difference from Recursively Enumerable (RE) Languages:

RE Languages: May loop forever on input that is not in the language.

Recursive Languages: Always halt.

Recursive languages are closed under complement, while RE languages are not.

An Undecidable Problem That is RE

The Halting Problem is the classic example of an undecidable problem that is Recursively Enumerable (RE).

Other Examples (also RE but undecidable)

- Acceptance problem A_{TM}

  The Acceptance Problem (often written as A_{TM}) is:

  A_{TM} = {(M, ω) | Turing Machine M accepts input w}

  In simple words: Given a Turing Machine M and a string w, does M accept w?
- Post Correspondence Problem (PCP)
- Modified Post Correspondence Problem (MPCP)

Halting Problem

The Halting Problem is the classic example of an undecidable problem that is Recursively Enumerable (RE). It asks whether a given Turing machine M halts on a specific input w. While no algorithm exists to decide this for all inputs, a Turing machine can simulate M on w, accepting if M halts, but potentially looping forever if M does not.

Formal Language Representation

HALT = {(M, W) | M halts on w}

- **Input** – A Turing machine and an input string **w**.
- **Problem** – Does the Turing machine finish computing of the string **w** in a finite number of steps? The answer must be either yes or no.
- **Proof** – At first, we will assume that such a Turing machine exists to solve this problem and then we will show it is contradicting itself. We will call this Turing machine as a **Halting machine** that produces a 'yes' or 'no' in a finite amount of time. If the halting machine finishes in a finite amount of time, the output comes as 'yes', otherwise as 'no'. The following is the block diagram of a Halting machine –

POST CORRESPONDENCE PROBLEM (PCP)

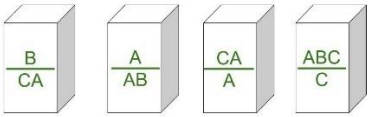
Post Correspondence Problem is a popular undecidable problem that was introduced by Emil Leon Post in 1946.

Undecidability of a problem means that there is no particular algorithm that can determine whether a given problem has a solution or not. A common undecidable problem is a Turing problem, so its un-decidability is also proved if we can reduce the Turing problem into post Correspondence problem.

In this problem we have N number of **Dominos** (tiles). The aim is to arrange tiles in such order that string made by Numerators is same as string made by Denominators. In simple words, lets assume we have two lists both containing N words, aim is to find out concatenation of these words in some sequence such that both lists yield same result.

Post Correspondence Problems can be represented in 3 ways:

1. Domino's Form :



2. Table Form :

	Numerator	Denominator
1	B	CA
2	A	AB
3	CA	A
4	ABC	C

3. List Form

X = { B, A, CA, ABC } Y= { CA, AB, A, C}

We are given two lists of strings over some alphabet.

Top list:

X = (X1, X2, X3,...,xn)

Bottom list:

Y = (У1, 11, Y2, Уз,..., Yn)

Each pair forms a tile (domino):

(X1/Y1), (X2/Y2),..., (Xn/Yn)

The problem is to determine whether there exists a sequence of indices
such that
i1, i2, i3, ... , ik
$Xi_1\ Xi_2$…. Xik = Yi Yi2….. Yik
That means the concatenation of top strings equals the concatenation of bottom strings.

Example-1

	A	B
1	1	111
2	10111	10
3	10	0

:

Explanation -

- **Step-1:** We will start with tile in which numerator and denominator are starting with same number, so we can start with either 1 or 2. Lets go with **second** tile, string made by numerator- 10111, string made by denominator is 10.
- **Step-2:** We need 1s in denominator to match 1s in numerator so we will go with **first** tile, string made by numerator is 10111 1, string made by denominator is 10 111.
- **Step-3:** There is extra 1 in numerator to match this 1 we will add **first** tile in sequence, string made by numerator is now 10111 1 1, string made by denominator is 10 111 111.
- **Step-4:** Now there is extra 1 in denominator to match it we will add **third** tile, string made by numerator is 10111 1 1 10, string made by denominator is 10 111 111 0.

  Final Solution - 2 1 1 3
  String made by numerators: 101111110
  String made by denominators: 101111110

Example 2
Find whether the lists **M = (ab, bab, bbaaa)** and **N = (a, ba, bab)** have a Post Correspondence Solution?
Solution

	x_1	x_2	x_3
M	ab	bab	bbaaa
N	a	ba	bab

In this case, there is no solution because − **| $x_2x_1x_3$ | ≠ | $y_2y_1y_3$ |** (Lengths are not same)
Hence, it can be said that this Post Correspondence Problem is **undecidable**.

Modified Post Correspondence problem

The Modified Post Correspondence Problem **(MPCP)** is a variation of the Post Correspondence Problem**)**

The sequence must start with the first tile (x1/y1)
After that, any tiles can be used (repetition allowed).
This is the main difference from Post Correspondence Problem, where the sequence can start with any tile.
We are given two lists of strings over some alphabet.
Top list:
X = (X1, X2, X3,...,xn)
Bottom list:
Y = (У1, 11, Y2, Уз,..., Yn)
Each pair forms a tile (domino):
(X1/Y1), (X2/Y2),..., (Xn/Yn)
The goal is to find a sequence of indices: 1,i2,i3,.., ik
such that:
X1 Xi2 Xi3…Xik=Y1 Yi2 Yi3…Yik

A language that is not recursively enumerable

A language that is not recursively enumerable (also known as non-Turing-acceptable or non-semidecidable) is a set of strings that cannot be recognized or generated by any Turing machine.

1. One example is the diagonal language, which is defined as the set of all strings that are not in the language recognized by a given Turing machine.
   L_d = {(M) | M does not accept (M)}
2. Other example is Complement of Acceptance Problem
3. One more standard example of a language that is **not recursively enumerable (not RE)** comes from the Halting Problem.
   Consider the language:
   HALT = {(M, W) | Turing Machine M does NOT halt on w}
   This is the complement of the Halting Problem**.**

COUNTER MACHINE

Counter Machine is a simple abstract computational model similar to a Turing Machine, but instead of using a tape it uses **counters (registers)** to store numbers.

Counter machine = A finite control + some integer counters.
A counter machine has:

- A finite set of states
- One or more counters
- Each counter stores a non-negative integer
- Simple operations on counters

Types of Counter Machines
1.One-Counter Machine

- Equivalent to Pushdown Automaton in power (roughly)
- Recognizes some non-regular languages

- But not fully Turing powerful

2 Two-Counter Machine

- Turing complete
- Can simulate a Turing Machine

Basic Operations

Typical instructions in a counter machine include:

1. Increment (INC):

   Increase the value of a counter by 1.
2. Decrement (DEC):

   Decrease the value by 1 (only if it's not zero).
3. Zero Test (JZ / IF ZERO):

   Check if a counter is zero and jump to another instruction.
4. Jump (GOTO):

   Move control to another instruction.